Universitat Autònoma de Barcelona

Aprenentatge Automàtic II

Detecció de Fake News mitjançant Deep Learning

Pol Ortiz Escobar

10 d'octubre de 2026

Índex

1 Selecció del Problema i Dataset **2**
1.1 Descripció del Dataset . . . 2
1.2 Justificació de la rellevància . . . 2

2 Anàlisi Exploratòria i Preprocessament **2**
2.1 Anàlisi Exploratòria (AED) . . . 2
2.1.1 Càrrega base de dades . . . 2
2.1.2 Distribució de la variable resposta . . . 2
2.1.3 Distribució del nombre de paraules . . . 3
2.1.4 Núvols de paraules . . . 3
2.2 Estratègies de transformació i Preprocessament . . . 4
2.2.1 Neteja del text (Cleaning) . . . 4
2.2.2 Tokenització i Vectorització . . . 4
2.2.3 Normalització de longitud (Padding) . . . 5
2.2.4 Partició del conjunt de dades . . . 5

3 Embeddings **5**
3.1 Introducció . . . 5
3.2 Arquitectura . . . 5
3.3 Entrenament i Aprenentatge de Representacions . . . 6
3.3.1 Actualització del Gradient i Co-ocurrència Estadística . . . 6
3.3.2 Eficiència Estructural . . . 6

4 Arquitectura Base: Multi-Layer Perceptron (MLP) **6**
4.1 Exploració i Justificació de l'Arquitectura . . . 6
4.2 Estratègies de Generalització i Regularització . . . 7
4.3 Entrenament i Diagnòstic . . . 7
4.4 Embeddings . . . 8

5 Arquitectures Avançades **9**
5.1 Justificació de l'arquitectura escollida . . . 9
5.2 Recurrent Neural Network (RNN) . . . 9
5.3 Long Short-Term Memory (LSTM) . . . 10
5.4 Entrenament i Diagnòstic . . . 10
5.5 Anàlisi comparatiu respecte a la MLP . . . 12

6 Comparació amb altres models d'aprenentatge **13**
6.1 Selecció del model clàssic: Regressió Logística amb TF-IDF . . . 13
6.2 Anàlisi comparatiu i discussió crítica . . . 13
6.3 Estratègia d'Ensemble: Soft Voting . . . 14

7 Avaluació final i Anàlisi d'Inferència **14**
7.1 Resultats sobre el conjunt de test . . . 14
7.2 Estudi de casos individuals . . . 15
7.2.1 Falsos positius . . . 15
7.2.2 Falsos negatius . . . 15

8 Conclusions **15**

9 Referències **16**

1 Selecció del Problema i Dataset

1.1 Descripció del Dataset

El dataset seleccionat prové de Kaggle (*Fake News Classification Dataset*), i conté titulars i textos de notícies amb una variable resposta que indica si la notícia és una "fake new"(notícia falsa) o vertadera. Totes les notícies del dataset són de mitjans de comunicació i temes dels Estats Units, per tant són totes en anglès i sobre temes majoritàriament relacionats amb el país. Per últim, cada observació de la base de dades és una notícia amb les variables: títol, text (contingut) i etiqueta (si és falsa o vertadera).

Per dades de text, on s'ha de fer un preprocessament del text per poder utilitzar-lo per entrenar el model, i on les dimensions de l'entrada són molt grans, solem tenir funcions altament no lineals i amb estructures molt complexes. Una xarxa neuronal ens permet dues coses: fer el preprocessament del text, ja que necessitem codificar el significat de les paraules d'una manera intel·ligent, amb capa d'embedding (això s'explicarà en detall més endavant en el treball), i capturar les estructures complexes de les dades.

1.2 Justificació de la rellevància

Amb la gran importància que han cobrat les xarxes socials per informar-se de les notícies del món, es necessita un mètode per poder filtrar aquestes grans quantitats de notícies en fiables i no fiables. Grans webs de notícies com **Reuters** ja utilitzen grans models de llenguatge per detectar possibles notícies falses que després són analitzades per persones, fent evident la necessitat de crear models d'aquest tipus.

2 Anàlisi Exploratòria i Preprocessament

2.1 Anàlisi Exploratòria (AED)

2.1.1 Càrrega base de dades

La base de dades té 24353 files sense cap NA (valor perdut), però tenint en compte el context del problema, podem assumir que una notícia de menys de 3 paraules no conté informació suficient per poder fer una predicció sobre la seva veracitat. També limitem a les notícies de menys de 4000 paraules per no sobrecarregar la memòria RAM i fer l'entrenament i validació del model més assequible. Després de filtrar per aquestes condicions queden 24273 observacions.

2.1.2 Distribució de la variable resposta

Per poder validar el model després de l'entrenament haurem d'utilitzar mètriques que poden ser millor o pitjors depenent de molts factors, entre ells la distribució de la variable resposta.

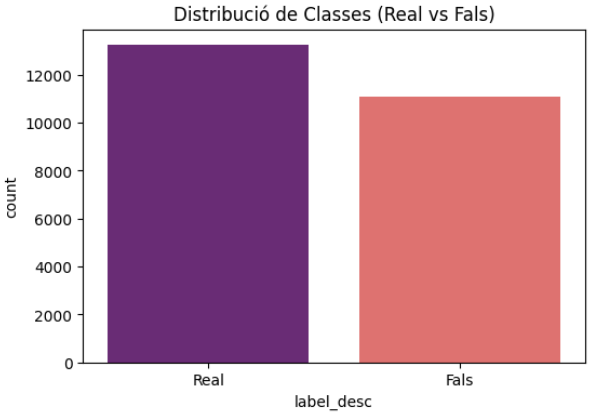


Figura 1: Distribució variable respota

Veiem que la variable resposta està una mica desbalancejada cap a les noticies reals, encara que aquesta diferència no és molt gran, caldrà observar tant l'accuracy com la F1 score i la matriu de confusió a la validació per asegurar que el model no està esbiaixat cap a la classe majoritària.

2.1.3 Distribució del nombre de paraules

Ara observarem la distribució del nombre de paraules per notícia en funció de si aquesta és real o falsa:

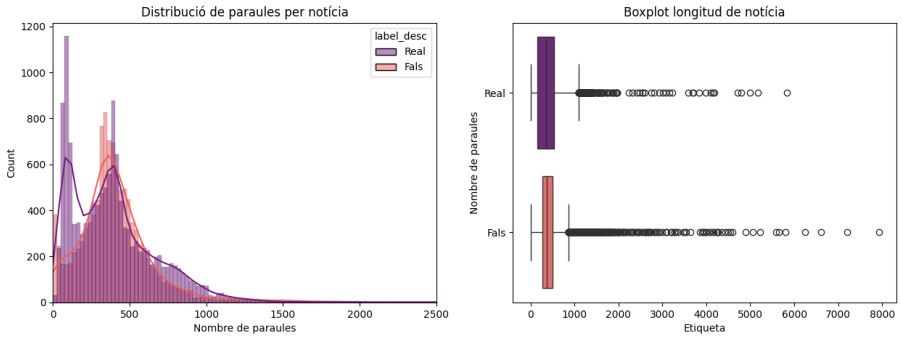


Figura 2: Distribució nombre de paraules en funció del tipus de notícia

Veiem que les dues classes tenen una distribució similar. En el primer gràfic hi ha un histograma del nombre de paraules en funció del tipus de notícia (limitat a 2500 paraules per veure millor la forma de la part més densa de les distribucions). Es veu com les notícies reals semblen tenir menys paraules en general encara que hi ha més variància. El més important ho observem al boxplot, on veiem que la gran majoria de les notícies se situen entre les 3 i 1500 paraules, on hi ha més outliers a les notícies falses. Com comentarem més endavant, al preprocessament de les dades haurem d'establir una longitud màxima de notícia que processarà el model (les que siguin més llargues es tallaran), per tant podem establir aquest límit a les 2000 paraules tenint en compte aquest gràfic.

2.1.4 Núvols de paraules

Per últim, és interessant veure la distribució de les paraules més frequents per cada classe, ja que es podria detectar visualment si algunes paraules tenen diferencies significatives de freqüència segns el tipus de notícia:

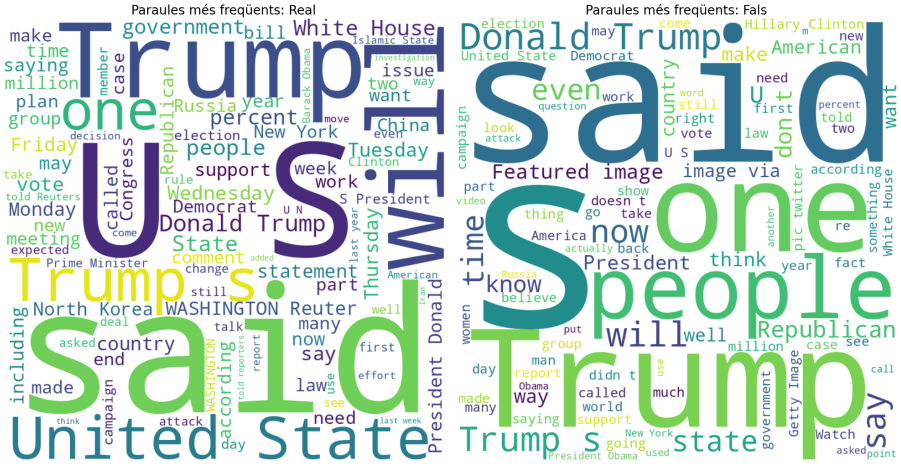


Figura 3: Núvol de paraules

Per cada gràfic, com més gran sigui la paraula, més alta és la seva freqüència. Veiem que no sembla haver-hi grans diferències en les paraules més utilitzades, el qual ens indica que és possible que les diferències entre notícies falses i reals no estiguin en les paraules individuals utilitzades, sinó en com es relacionen entre si.

2.2 Estratègies de transformació i Preprocessament

El preprocessament és una fase crítica en el disseny de xarxes neuronals per a text, ja que els models de *Deep Learning* no poden processar dades en brut. El flux de treball implementat es divideix en quatre etapes principals:

2.2.1 Neteja del text (Cleaning)

S'ha aplicat una funció de neteja personalitzada per reduir el soroll i la variabilitat del vocabulari. Les accions realitzades inclouen:

- **Normalització:** Conversió de tot el text a minúscules per evitar que el model consideri la mateixa paraula com a diferent segons la seva capitalització.
- **Eliminació de soroll:** S'han utilitzat expressions regulars (*Regex*) per suprimir URLs, etiquetes HTML i caràcters especials o de puntuació.
- **Filtratge de *stop-words*:** Mitjançant la llibreria `nltk`, s'han eliminat les paraules funcionals de l'anglès (com *"the"*, *ïs"*, *"at"*) que, tot i ser freqüents, no aporten valor semàntic per diferenciar una notícia real d'una falsa.
- **Eliminació informació 'trampa':** S'ha eliminat el nom de l'agència de notícies 'Reuters' ja que apareix molt a les notícies reals. Això és perque la xarxa no aprengui a classificar com a real qualsevol notícia on aparegui '(Reuters) -' (o similars) i com falses les que no ho continguin. Ja que les dades provenen tant de diaris com de blogs individuals, normalment les notícies reals contindran aquest nom i les falses no. Com es vol que la xarxa classifiqui en funció de l'estructura del text, s'han eliminar aquestes expressions 'trampa' que donarien lloc a un model molt simple.

2.2.2 Tokenització i Vectorització

Un cop net el text, s'utilitza la classe `Tokenizer` de Keras per transformar les paraules en seqüències numèriques. Aquest procés funciona de la següent manera: es limita el diccionari a les

10.000 paraules més freqüents (`max_words`). Això permet que el model es centri en els termes amb més pes estadístic. Llavors a cada paraula se li associa un nombre de l'1 al 10.000, menys a les que no entren al diccionari, a les quals associem l'etiqueta 'OOV', que és estandard per designar les paraules desconegudes. Amb això fet, l'objectiu és poder representar numèricament el significat d'una paraula, ja que amb un sol nombre no tenim suficient informació. Això es fa amb la capa d'embeddings, que transforma cada paraula associada a un nombre en un vector d'altes dimensions, on cada dimensió acaba representant significats abstractes com "sexe"o "plural/singular".

2.2.3 Normalització de longitud (Padding)

Les xarxes neuronals requereixen una entrada de dimensió fixa. Per tant hem definit basant-nos en l'AED, un llindar de 1500 paraules de longitud a les noticies, llavors els vectors amb longituds diferents al llindar es tallaran o ompliran de 0's per arribar a la mida desitjada. D'aquesta forma totes les noticies tindran les mateixes dimensions.

2.2.4 Partició del conjunt de dades

Seguint les bones pràctiques de validació creuada i per garantir una avaluació imparcial del model, s'han utilitzat 3 conjunts de dades separats:

- **Entrenament (24353 obs.):** Utilitzat per a l'ajust dels pesos i la matriu d'embeddings.
- **Validació (8117 obs.):** Utilitzat per monitorar el procés d'entrenament, ajustar hiperparàmetres i detectar *overfitting* de manera prematura.
- **Test (8117 obs.):** Reservat per a l'avaluació final de les mètriques.

3 Embeddings

3.1 Introducció

Els embeddings són una forma de fer entendre a les xarxes neuronals el significat de les paraules a través d'una representació numèrica. Aquesta representació consisteix en associar cada paraula del diccionari mencionat abans, a un vector d'altes dimensions, on cada dimensió volem que representi conceptes semàntics abstractes. D'aquesta manera paraules amb significats similars tindran vectors similars (en direcció i posició).

3.2 Arquitectura

Per aconseguir això afegim al principi de la xarxa una capa d'embeddings amb matriu de pesos $E \in \mathbb{R}^{V \times D}$, on D és la dimensió de l'espai on viu el vector de cada paraula, i V la mida del diccionari que hem definit. Aquesta capa no es tracta com una normal, sino que al passar l'input d'un text convertit en tokens $x \in \mathbb{Z}^N$, on N és la longitud del text, per cada token es busca la columna de la matriu en la posició igual al seu valor, i es converteix aquest token en la columna corresponent. Així al passar una entrada d'un text en forma de vector per la matriu d'embeddings, obtenim una matriu $z_{emb} \in \mathbb{R}^{N \times D}$ on cada fila és una paraula representada en l'espai d'embeddings.

Això s'aplicarà al cas d'una MLP. Però en general en comptes d'una única capa, els embeddings segueixen el mateix concepte però am una xarxa neuronal completa que pot tenir molt més d'una capa, on la surtida és el vector de dimensió D.

3.3 Entrenament i Aprenentatge de Representacions

En el cas d'embeddings més avançats s'entrena una xarxa neuronal amb objectius de predir la següent paraula d'un text, d'aquesta forma la xarxa aprén la semàntica de les paraules en general i no només per un objectiu en específic.

En el cas de la MLP d'aquest treball, l'aprenentatge dels valors de la matriu E no és un procés aïllat, sinó que es realitza mitjançant l'algorisme de *backpropagation* que minimitza la funció de pèrdua $\mathcal{L}$. Encara que l'objectiu de la xarxa és la classificació i s'optimitza per millorar aquesta tasca, la matriu d'embeddings acaba capturant relacions semàntiques degut a dos mecanismes matemàtics fonamentals:

3.3.1 Actualització del Gradient i Co-ocurrència Estadística

Durant cada iteració de l'entrenament, per a una mostra d'entrada x, el gradient de la pèrdua respecte a la matriu d'embeddings, $\frac{\partial \mathcal{L}}{\partial E}$, només presenta valors diferents de zero per a les files corresponents als tokens que apareixen en el text.

Si dues paraules w_i i w_j apareixen sovint en contextos similars, els seus vectors respectius e_i i e_j rebran actualitzacions de gradient en direccions correlacionades. Aquest fenomen desplaça els vectors cap a una regió comuna de l'espai $\mathbb{R}^D$ per permetre que les capes posteriors puguin aplicar operacions lineals consistents sobre conceptes anàlegs. Matemàticament, els pesos convergeixen cap a uns valors on la **similitud del cosinus** entre paraules funcionalment equivalents tendeix a 1:

$$\text{sim}(e_i, e_j) = \frac{e_i \cdot e_j}{\|e_i\|\|e_j\|} \approx 1$$

3.3.2 Eficiència Estructural

La xarxa tendeix a cercar la solució amb la mínima complexitat estructural possible. Si el model assignés vectors ortogonals o distants a paraules amb el mateix rol predictiu, les capes subsegüents haurien d'aprendre hiperplans de decisió extremadament fragmentats per arribar a la mateixa conclusió.

En agrupar paraules similars en el mateix lloc de l'espai, el model redueix la variància de les activacions i facilita una convergència més estable cap a un mínim global.

4 Arquitectura Base: Multi-Layer Perceptron (MLP)

4.1 Exploració i Justificació de l'Arquitectura

Per processament de text, com el nostre cas, models clàssics no solen tenir un molt bon rendiment, ja que les relacions entre el text y la variable respota poden ser altament no lineals i de moltes dimensions. Les xarxes neuronals son el model perfecte per capturar aquestes relacions no lineals i poder processar la relació del text amb la variable resposta. En l'arquitectura base utilitzem una MLP (fully connected) amb una capa d'embeddings per poder processar el text.

- **Profunditat i Amplada:** Per dades amb relacions complexes com pot ser un bloc de text, és necessaria una alta qualitat de representació, és a dir, un bon nombre de regions a la funció resultant. Per això s'ha d'escollir bé tant el nombre de neurones com la seva distribució en capes. Per un diccionari relativament petit (10000 paraules) i una predicció de només dues classes, no serà necessari un gran nombre de neurones, però després de fer proves amb una o dues capes, la xarxa amb dues capes té un millor rendiment. En quant al nombre de neurones amb 64 neurones a les capes intermitges sembla ser suficient per arribar a tenir un bon model.

- **Funcions d'Activació:** Utilitzarem la `ReLU` per les capes intermitges ja que ajuda a combatre el problema del *banishing gradient* i és menys costosa d'entrenar en comparació a una `Sigmoide`. I per la capa de sortida ja que volem classificar en 2 classes utilitzarem una `Sigmoide`.
- **Configuració final:** La combinació escollida consisteix en una capa d'*Embedding* ($E \in \mathbb{R}^{V \times D}$), una capa de *GlovalAveragePooling* que fa la mitjana, i dues capes denses (128 i 64 neurones) seguint un criteri d'embundament d'informació. És important destacar la capa de *GlovalAveragePooling* ja que farà que la xarxa tracti les paraules de les notícies com observacions independents. No s'ha utilitzat una capa *Flatten* ja que provocaria una explossió de paràmetres al necessitar una capa amb moltíssimes neurones per ser compatible amb l'entrada del vector de 1500 x 100 = 150.000 components.
- **Paràmetres:** El model final té un total de 1.000.000 de paràmetres a la capa d'embeddings i 21.249 a la resta de capes. S'obtenen un total d'**1.021.249 paràmetres** entrenables (no s'estan tenint en compte els paràmetres de l'optimitzador).

4.2 Estratègies de Generalització i Regularització

Per tal d'evitar l'overfitting del model s'han utilitzat dues estratègies de regularització, una implícita i una explícita:

1. **Early stopping:** Es fixa una paciència i a l'entrenament, per cada època, es mira si l'accuracy al dataset de test ha baixat. Si l'accuracy porta baixant 3 èpoques seguides, es para l'entrenament i s'agafen els pesos d'abans que comencés a baixar.
2. **Weight Decay (L_2):** S'ha integrat una penalització en la funció de pèrdua per limitar la magnitud dels pesos. Això ajuda a donar més importància a paraules poc comuns que podrien tenir poc pes, però que poden ser decissives a l'hora d'entendre un text. La noca actualització del descens del gradient és:

$$\Theta_{t+1} = (1 - \eta\lambda)\Theta_t - \eta\nabla L_{data}$$

On λ és el factor de regularització, i s'ha utilitzat $\lambda = 0.01$.

4.3 Entrenament i Diagnòstic

S'ha entrenat un model MLP sense regularització i un altre amb regularització per evaluar si és necessari l'ús d'aquests mètodes. L'entrenament d'ambdós models es resumeix en aquest gràfic:

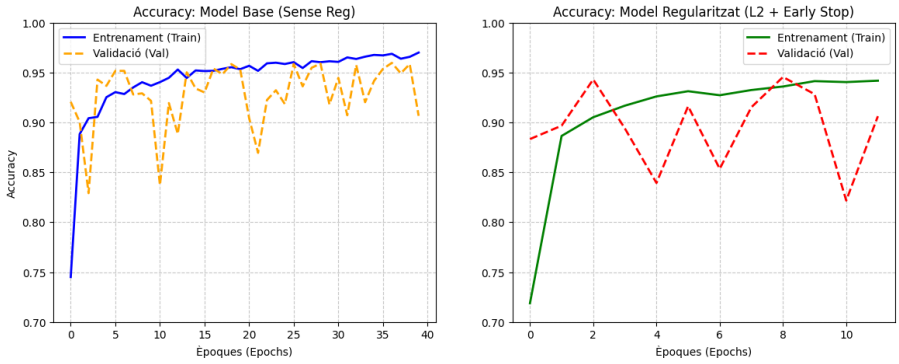


Figura 4: Corves d'aprenentatge models MLP amb i sense regularització

Veiem que els dos models tendeixen a augmentar l'accuracy per cada època, però al MLP sense regularització la corva de l'accuracy en les dades de test comença a ser més erràtica cap a

les èpoques finales fins i tot baixant a lúltima, mentres que l'accuracy a les dades d'entrenament creix de manera quasi constant arribant a valors molt alts. Això és un clar exemple d'overfitting, ja que d'haber parat l'entrenament unes èpoques abans la accuracy a dades no observades pel model seria bastant més alta. En el cas del model amb regularització veiem com l'early stopping ha parat l'entrenament a la època 18. Així obtenint una accuracy a dades de text més alta.

Taula 1: Comparativa de les mètriques de classificació en el conjunt de test (Model Base vs. Regularitzat)

Model	**Classe / Mètrica**	**Precisió**	**Recall**	**F1-Score**	**nº obs.**
Model Base	Fals	0.85	0.99	0.91	3753
	Real	0.99	0.85	0.91	4364
	Accuracy		**0.91**		8117
Model Regularitzat	Fals	0.93	0.98	0.95	3753
	Real	0.98	0.93	0.96	4364
	Accuracy		**0.95**		8117

4.4 Embeddings

Després de realiztar l'entrenament dels dos models, obtenim unes matrius d'embeddings molt rudimentàries però que haurien d'haver capturat algún petit significat semàntic de les paraules del diccionari format per la base de dades. Podem intentar visualitzar això mitjançant el mètode **t-sne**[1]. Aquest és un mètode de reducció de dimensionalitat no lineal que utilitza algorismes probabilístics per reduir le dimensió d'un conjunt de dades intentant mantenir propers els punts que són propers a l'espai d'altes dimensions. En matrius d'embeddings on les relacions semàntiques entre les paraules poden tenir estructures àltament no lineals, aquest mètode és bastant millor que **PCA**, qe redueix la dimensió mantenint el màxim possible les distàncies euclidianes, per tant és un mètode lineal.

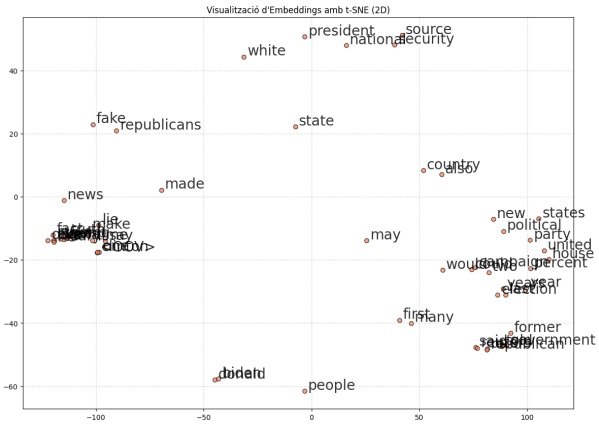


Figura 5: Visualització 2D embeddings model MLP amb regularització

Veiem que paraules com 'president' i 'national' són a prop. Cosa que té sentit semànticament en el context de la base de dades.

5 Arquitectures Avançades

5.1 Justificació de l'arquitectura escollida

En aquest model es faran dos canvis: un a la matriu d'embeddings, i un altre a l'arquitectura de la xarxa.

En primer lloc, la matriu d'embeddings ara no s'entrenarà amb la base de dades d'aquest treball ni juntament amb la xarxa. S'utilitzará una matriu d'embeddings ja entrenada per separat. Així es podría dir que la xarxa no ha d'apendre a 'llegir', ja sap. Ara només haurà d'apendre a entendre que diuen aquestes paraules sobre la veracitat de la notícia. La matriu que s'utilitzará es la de **Glove**[2] creada per *Standford Natural Language Processing Group* i entrenada sobre *Wikipedia*. Concrètament la de dimensió 100 de l'espai d'embeddings.

A part d'això, el problema principal de l'arquitectura del MLP bàsic és que tracta les variables d'entrada com observacions independents, ja que cada paraula és un vector i la única forma de pasar una notícia completa a la xarxa sense fer una explossió de paràmetres (amb la capa *Flatten()*)és fer una mitjana d'aquest vector amb la capa GlobalAveragePooling1D(). A l'anàlisi de text es vol tractar las paraules com variables relacionades entre sí per poder entendre el context per cada paraula i així poder saber el que s'està expresant. Per resoldre aquest problema es pot utilitzar una arquitectura de xarxa neuronal que tingui en compte l'ordre de les variables per fer una predicció.

5.2 Recurrent Neural Network (RNN)

Una bona opció seria una **RNN**[8], que modela la relació temporal entre les variables d'entrada $x_1, \ldots, x_t, \ldots, x_n$. Aquesta arquitectura combina moltes xarxes neuronals d'una capa (poden ser deep i es tindria una **DeepRNN**), on comennçant per la observació més 'antiga' (en aquest cas la primera paraula del text) la pasa per una d'aquestes xarxes, i a cada paraula seguent, l'input és una combinació de les sortides anteriors amb la actual. Aquesta combinació es fa amb una matriu de pesos i una transformació no lineal. Al final es calcula la predicció amb el vector final que és combinació de totes les variables amb una MLP normal (en aquest cas SÍ que pot ser deep).

Matemàticament, l'evolució d'aquest estat intern (o memòria) en un instant de temps t es pot descriure com:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \tag{1}$$

On h_t és l'estat ocult actual (la sortida d'aquest pas temporal), x_t és el vector de la paraula actual de la seqüència, i h_{t-1} és l'estat ocult acumulat del pas temporal anterior. L'aspecte clau d'aquesta equació és el principi de compartició de paràmetres: les matrius de pesos W_{hh} (que operen sobre la memòria) i W_{xh} (que operen sobre l'entrada), així com el vector de biaix b_h, són exactament les mateixes iteració rere iteració. La funció no lineal aplicada sol ser la tangent hiperbòlica (tanh).

Però aquesta arquitectura té 2 principals problemes molt relacionats: per seqüències de variables molt grans, la influència de les primeres variables sobre la sortida és o molt petita o molt gran depenent de la matriu de pesos de la recurrència, i gràcies a que passa això, el gradient pot ser o molt influenciat o molt poc per aquestes variables.

Podem entendre el problema del gradient observant que per calcular el gradient de la funció de pèrdua L respecte a un estat inicial (com h_1), cal aplicar la regla de la cadena de forma

iterativa al llarg de tota la seqüència de longitud T:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} \tag{2}$$

I com el primer terme és $\frac{\partial h_t}{\partial h_{t-1}} = W_{hh}\text{diag}(\tanh'(\cdot))$, el gradient global requereix multiplicar la matriu de pesos per si mateixa $T-1$ vegades. Simplificant-ho:

$$\frac{\partial L}{\partial h_1} \propto (W_{hh})^{T-1} \tag{3}$$

I en aquesta relació exponencial es veu el problema: si els valors propis (*eigenvalues*) de la matriu W_{hh} són menors que 1 (completament habitual a causa de l'acotació de la derivada de tanh), el terme $(W_{hh})^{T-1}$ tendeix exponencialment a zero. Al afegir aquest terme al gradient total, l'aportació és mínima.

5.3 Long Short-Term Memory (LSTM)

La solució a aquest problema es una arquitectura **LSTM**[3], que tracta dirèctament amb el problema del gradient. Aquesta en termes d'ús és molt similar a una RNN, ja que les dues volen modelar la relació temporal d'unes variables d'entrada, encara que la seva arquitectura és bastant més complexa. En aquesta arquitectura es té una memòria a llarg termini, que guarda patrons que s'han considerat rellevants de les primeres variables, i una memòria a curt termini, que dona importància a les variables més properes a la última.

La solució que dona al problema del gradient, sense entrar molt en detall, és que el valor de la memòria a llarg termini s'actualizta amb sumes, en comptes de multiplicacions com a les RNN, d'aquesta manera es soluciona el problema del gradient ja que ara s'actualitza aditivament en comptes de multiplicativament, i de la importància de les primeres variables.

Entrant en detalls més concrets de l'arquitectura:

- **Configuració final**: El model consisteix en una capa d'*Embedding*, seguida d'una capa *LSTM* bidireccional (analitza de principi final y de final a principi) de 64 neurones amb regularització *L2* (amb paràmetre de regularització 0.01), segguida d'una capa de Densa amb funció d'activació ReLU i la mateixa regularització amb 32 neurones, i una capa final per classificar amb funció d'activació Sigmoide.

- **Paràmetres:** El model final té un total de 1.000.000 de paràmetres a la capa d'embeddings i 88.641 a la resta de capes. S'obtenen un total d'**1.088.641** paràmetres entrenables (de nou sense tenir en compte els paràmetres de l'optimitzador).

Amb aquesta arquitectura es podria dir que la capa LSTM i els embeddings s'encarreguen d'extreure tota la informació rellevant de la notícia i de ordenar-la per poder passar un vector que conté tota aquesta informació a una xarxa MLP bastant senzilla que pugui classificar aquests vectors. Aquesta última xarxa no ha de ser molt complexa ja que tot el treball d'extracció de característiques importants de la notícia ja està fet, ara només cal classificar un vector en un espai que pot arribar a ser casi linealment separable. Afegir una xarxa molt gran al final pot arribar a causa overfitting ja que està tractant un problema que ha perdut molta complexitat.

5.4 Entrenament i Diagnòstic

L'entrenament s'ha fet en dues etapes:

- **Primera etapa:** En aquesta es congelen els pesos de la matriu d'embeddings i s'entrenan els pesos de tota la resta de la xarxa. A part, s'ha aplicat, a més de la regularització *L2* ja present en l'estructura de la xarxa, una regularització implícita d'**early stopping**.

- **Segona etapa:** Consisteix en el **Fine Tunning** de la matriu d'embeddings. S'ha descongelat la capa d'embeddings i s'han entrenat 3 èpoques amb un *learning rate* molt baix per canviar una mica però no molt els embeddings. D'aquesta forma el significat semàntic de les paraules encaixa una mica més amb aquest dataset en concret.

S'han obtingut les següents corves d'aprenentatge:

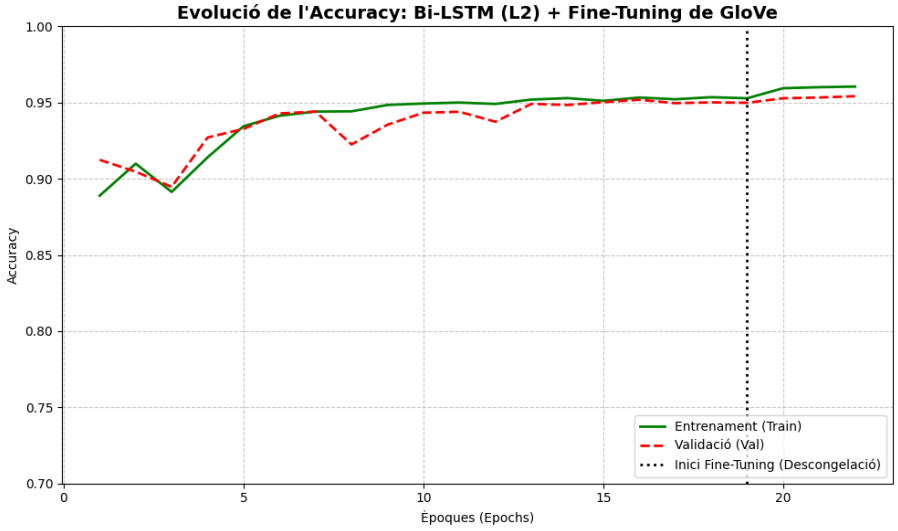


Figura 6: Corves d'aprenentatge model LSTM

Viem que les corves d'accuracy als conjunts d'entrenament i de test son creixents. A més, no sembla haver presència d'overfitting ja que l'accuracy dels dos conjunts al final de la corva és molt similar. S'han realitzat 19 de les 30 èpoques especificades al codi per la primera part de l'entrenament, on l'*early stopping* ha detectat que l'accuracy al set de validació estaba baixant i ha parat l'entreneament conservant els pesos de la millor època (en quant a rendiment. En la segona etapa de *fine tunning* l'accuracy a augmentat lleugerament tant al conjunt d'entrenament com al de test. S'han obtingut les següents mètriques de validació:

Taula 2: Mètriques de classificació en el conjunt de test per al model LSTM

Model	**Classe / Mètrica**	**Precisió**	**Recall**	**F1-Score**	**nº obs.**
LSTM	Fals	0.95	0.97	0.96	3753
	Real	0.97	0.95	0.96	4364
	Accuracy		**0.96**		8117

Per últim podem visualitzar els embeddings obtinguts amb **t-sne**:

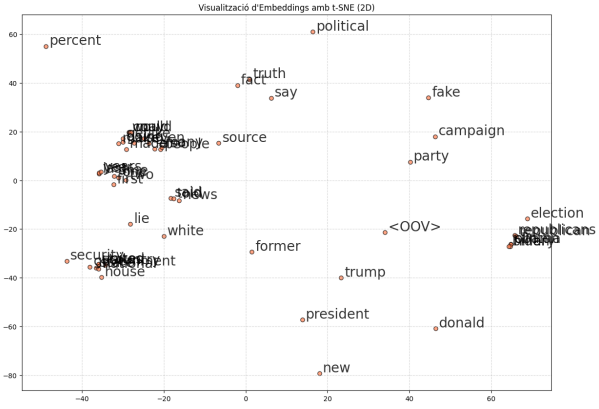


Figura 7: Visualització 2D embeddings Glove x LSTM

Es pot observar que hi han clusters de paraules molt més pronunciats que als embeddings de la MLP. I també es poden veure relacions amb sentit com 'president', 'trump' i 'donald' més o menys a prop.

5.5 Anàlisi comparatiu respecte a la MLP

Observant les mètriques d'accuracy, veiem que només hi ha un augment d'un 1% respecte la MLP amb regularització. Aquesta diferència és molt petita i donada la complexitat del model LSTM amb embeddings preentrenats, podem pensar que el model MLP és una millor opció ja que en rendiment és pràcticament igual, el model és més senzill en quant a funcionament i el temps d'entrenament és bastant menor (2,4353m vs 63,66m), a part d'això el nombre de paràmetres és bastant similar amb 1.021.249 i 1.088.641 paràmetres de la MLP i LSTM respectivament.

Però per què sabem que no funciona el model LSTM? Podem fer dues hipòtesis de com estan funcionant els dos models per predir la probabilitat de que una notícia sigui real o falsa:

- **MLP**: Com pren les variables com observacions independents, dona igual l'ordre de les paraules a la notícia amb la que alimentem la xarxa, sempre hauria de donar les mateixes prediccions per les mateixes paraules. D'aquesta forma es pot intuir que la xarxa està fent prediccions en funció de quan associades estan certes paraules a notícies reals o falses (evidentment aquest procés no té perquè ser una simple ponderació de freqüències relatives, segurament té una naturalessa molt menys lineal i més complexa).

- **LSTM**: L'objectiu d'aquesta xarxa és modelar les relacions entre paraules, i es voldria que pogués classificar notícies tenint en compte no només l'associació de les paraules a les determinades classes, sinó també l'estructura del text (si és més sensacionalista, més professional, etc). Per avaluar això podem pasar la mateixa notícia per la xarxa però canviant l'ordre de les paraules per donar-li una estructura diferent:

  - **Real**: 'The Pentagon confirmed that it is a completely false rumor about an imminent attack. No troops are going to be deployed.' → **Predicció:** FALS (63,84%)
  - **Falsa**: 'The Pentagon confirmed a rumor about an imminent attack. It is completely false that no troops are going to be deployed.' → **Predicció:** FALS (67,18%)

  Es veu que si que té més confiança en la notícia modificada per semblar més falsa, però no un valor significativament superior a la original suposadament real. Per tant no sembla que la xarxa hagi aprés a entendre el context del les paraules ja que per estructures diferents dona prediccions molt similars.

Llavors es pot pensar que la MLP és un millor model per aquest problema.

6 Comparació amb altres models d'aprenentatge

6.1 Selecció del model clàssic: Regressió Logística amb TF-IDF

Com a model clàssic s'ha seleccionat un model de **Regressió Logística** combinat amb una vectorització de text mitjançant **TF-IDF**[4] (*Term Frequency - Inverse Document Frequency*).

En els models anteriors s'han pogut utilitzar embeddings, però a un model de regressió logística no és la millor opció ja que no es poden utilitzar els vectors d'embeddings dirèctament però tampoc és bona idea fer la mitjana de tots els vectors d'una notícia ja que no és molt bona representació de la informació. Així que s'utilitza una mesura de la freqüència de les paraules al text en relació amb la freqüència amb que apareix a les notícies en tota la base de dades:

$$\text{TF-IDF}_{i,j} = \text{TF}_{i,j} \times \log\left(\frac{N}{\text{DF}_i}\right) \tag{4}$$

On $\text{TF}_{i,j}$ és la freqüència del terme en el document, N és el nombre total de notícies del corpus i DF_i és el nombre de documents que contenen la paraula.
Intuitívament, el primer terme mesura la importància de la paraula a la notícia que s'està predint, i el segón mesura si aquesta paraula és molt comú a tota la base de dades, ja que si una paraula apareix sempre a les notícies de la base de dades, no aporta molta informació que aparegui també a la notícia que estem tractant, així que la divisió és ≈ 1 i per tant el logaritme és molt proper a 0, així no li dona importància a aquesta paraula. En canvi, per paraules menys freqüents a la base de dades el segón terme augmenta i si la paraula té importància a la notícia el valor augmenta.

Posteriorment, la Regressió Logística modela la probabilitat de falsedat mitjançant la funció sigmoide aplicada a una combinació lineal de les freqüències:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{5}$$

6.2 Anàlisi comparatiu i discussió crítica

S'entrena i valida el model amb els mateixos conjunts de dades que s'han utilitzat per les xarxes neuronals. I s'obtenen les següents mètriques:

Taula 3: Mètriques de classificació en el conjunt de test per al model clàssic

Model	**Classe / Mètrica**	**Precisió**	**Recall**	**F1-Score**	**nº obs.**
Regressió Logística	Fals	0.95	0.96	0.95	3803
	Real	0.96	0.96	0.96	4314
	Accuracy		**0.96**		8117

Utilitzant aquestes mètriques, es pot dividir l'anàlisi comparatiu en 3 apartats:

- **Temps de computació i optimització:** La Regressió Logística presenta un avantatge molt gran ja que el temps d'entrenament és molt inferior al de la MLP.
- **Complexitat paramètrica:** La dimensionalitat del model clàssic està limitada al tamany del vocabulari filtrat (10.000 coeficients), i per el funcionament del model es pot saber d'on venen exactament les prediccions ja que és un model de caixa blanca, al contrari que la MLP.

- **Funcionament:** S'observa que el model clàssic assoleix un rendiment molt bó, de fet millor que el de la MLP per un 1%. Aquest fenomen confirma la hipòtesi de com fa les prediccions la MLP, és a dir, la distribució de paraules clau i adjectius sensacionalistes és tan diferent entre ambdues classes que un classificador lineal com la regressió logística és capaç de traçar una frontera de decisió basant-se en la presència de paraules aïllades, sense trobar patrons més complexos. Una solució a la que probablement ha arribat també la MLP.

Vist això queda clar que el millor model per aquest problema serà una Regressió Logística ja que dona el mateix rendiment amb una complexitat del model molt menor.

6.3 Estratègia d'Ensemble: Soft Voting

Es pot implementar un ensemble de tipus **Soft Voting**. Aquesta estratègia fusiona les prediccions de dos models utilitzant les probabilitats que prediuen. En aquest cas, encara que s'ha conclós abans que el MLP era millor que el LSTM, com té un funcionament molt semblant al model logístic, és preferible combinar les prediccions del LSTM i la regressió logística, ja que son dos models més diferents en funcionament i en prediccions. La idea és que el LSTM tingui més en compte el context i el significat de la notícia (encara que no tingui un molt bon funcionament en aquest aspecte), i la regressió logística tingui en compte la importància de les paraules individualment per la predicció.

Matemàticament, la predicció es fa com:

$$P_{\text{Ensemble}}(y = 1|x) = \alpha \cdot P_{\text{MLP}}(y = 1|x) + (1 - \alpha) \cdot P_{\text{RegLog}}(y = 1|x) \tag{6}$$

On s'ha avaluat l'accuracy al conjunt de validació de tots els valors de α del 0 a l'1 amb separació de 0.05 obtenint un valor òptim de $\alpha = 0.3$. S'han obtingut els resultats següents:

Taula 4: Mètriques de classificació en el conjunt de test per a l'Ensemble

Model	Classe / Mètrica	Precisió	Recall	F1-Score	nº obs.
Ensemble	Fals	0.96	0.97	0.97	3803
	Real	0.97	0.97	0.97	4314
	Accuracy		**0.97**		8117

7 Avaluació final i Anàlisi d'Inferència

7.1 Resultats sobre el conjunt de test

S'utilitza com a model final l'ensemble del model LSTM i la regressió logística. I s'obtenen les següents mètriques:

Taula 5: Mètriques de classificació en el conjunt de test per al model Ensemble final

Model	Classe / Mètrica	Precisió	Recall	F1-Score	nº obs.
Model Final	Fals	0.97	0.97	0.97	3753
	Real	0.98	0.97	0.97	4364
	Accuracy		**0.97**		8117

Amb un accuracy de 0.97 i altres mètriques iguas o millors, es pot veure clarament que és model amb millor comportament predictiu fins ara. Encara que fer un k-fold cross validation donaria unes mètriques més robustes i fiables, entrenar k vegades aquest mateix ensemble trigaria

molt temps així que es prenen aquestes mètriques com una bona aproximació del comportament del model.

7.2 Estudi de casos individuals

Per analliztar com es comporta el model és necessari analitzar els casos en que s'ha equivocat per veure si té sentit segons el que s'ha dit prèviament sobre com hauria de comportar-se el model.

7.2.1 Falsos positius

- **WAKE UP! GEORGE SOROS "Drastically Ramps Up" Effort To Destroy America**:
  "George Soros is trying his best to destroy America! He is ramping up his efforts now! James Woods said it best when he tweeted that Soros would be happy with what happened in Charlottesville: Attorney General Jeff Sessions released..."

  Aquesta notíca ha sigut classificada com a real amb una probabilitat d'un 56.05%. Al principi pot semblar una noticia clarament uan notícia falsa per les exclamacions i el tó sensacionalista i conspiranoic, però en llegir tota la noticia es pot observar que més endavant pren un tó molt més seriós i periodístic, on s'utilitzen dades financieres i vocabulari institucional. Aquesta segona part pot haber empeyat la predicció lo suficient cap a real fins arribar a fer que la noticia sigui classificada com a real.

7.2.2 Falsos negatius

- **Bank fines elect democrats**:
  "the psychopathic and insincere jihad islams natural appeal to criminals psychopaths and murderers october raymond ibrahim is a shillman fellow at the david horowitz freedom center whenever muslims engage in behavior that ostensibly contradicts islamfrom taking drugs an watching porn to killing fellow muslims islams apologists..."

  Aquesta notícia semblar clarament falsa o com a mínim sensacionalista. El principi de la noticia és molt extrems, contenint paraules com 'psicòpates'. Això probablement ha fet que el model predigui la notícia com falsa. Aquesta notícia probablement prové d'un apartat d'opinió d'un medi de comunicacó de notícies, i a la base de dades ha sigut classificat com real, però clarament no és una notícia i per això el model la ha classificat incorrectament.

8 Conclusions

Al llarg d'aquest treball he après a construir, validar i utilitzar models de deep learning, amb arquitectura MLP i LSTM. A part de construir-los, també he après el seu funcionament, començant per les MLP, intentant tractar amb variables relacionades amb les RNN i finalment solucionant els problemes d'aquesta última arquitectura amb la LSTM. A part d'això una part fonamental d'aquest treball ha sigut el tractament de dades de text per fer models predictius amb elles i poder interpretar com les veuen aquests models amb els embeddings.

El model final és una combinació d'una arquitectura de xarxa neuronal LSTM i una regressió logística mitjançant un ensemble de soft voting. Aquest model té la intenció de fixar-se principalment en les paraules utilitzades a les notícies per fer la predicció, i en menor mesura

en l'estructura de la notícia per empènyer la predicció cap a un costat o un altre segons si l'estructura és més periodística/professional o més sensacionalista. I amb aquesta estructura s'han aconseguit molt bones mètriques a un conjunt de dades no vist pel model, per tant es pot pensar que té una bona capacitat de generalització.

Com s'ha comentat a un apartat anterior, el model MLP és preferible per si sol al model LSTM, ja que té unes mètriques molt similars, però sent un model molt més simple i eficient computacionalment. El problema és que un aspecte clau per un model amb l'objectiu de predir si una notícia és falsa o no en funció del seu contingut i sense accés a eines per verificar la veracitat d'aquest contingut, ha de tenir algun mecanisme per tenir en compte l'estructura del text, i deixar un model final sense aquesta capacitat s'ha considerat com un error.

Com a limitació del model, clarament està la seva capacitat per entendre l'estructura del text, que s'ha comprovat que no és molt bona i no aporta molt a les prediccions (assumint que existeix). Per tant una possible millora seria implementar una arquitectura que sí pogués entendre l'estructura d'un text, o trobar una forma d'entrenar la LSTM perquè pugui aprendre aquests patrons. Una arquitectura candidata serien els transformers, utilitzats a tots els chatbots moderns com *ChatGPT* o *Gemini*. Tot i que aquests tenen un cost computacional encara més alt, amb un bon volum de dades i temps d'entrenament poden tenir molt bons resultats.

9 Referències

Referències

[1] Van der Maaten, L., & Hinton, G. (2008). *Visualizing Data using t-SNE*. Journal of Machine Learning Research, 9, 2579–2605.

[2] Pennington, J., Socher, R., & Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation*. En Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 1532–1543).

[3] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.

[4] Salton, G., & Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, 24(5), 513–523.

[5] Scikit-learn developers. *sklearn.manifold.TSNE*. Documentació oficial de l'API. Recuperat de: `https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html`

[6] TensorFlow. *tf.keras.layers.LSTM*. Documentació oficial de l'API. Recuperat de: `https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM`

[7] GeeksforGeeks. *Text Classification using Logistic Regression*. Recuperat de: `https://www.geeksforgeeks.org/machine-learning/text-classification-using-logistic-regression/`

[8] Sensio, J. *Introducción a las Redes Neuronales Recurrentes (RNN)*. Blog personal. Recuperat de: `https://www.juansensio.com/blog/034_rnn_intro`